

ClassRoomPB

Sistema Acadêmico — ES2

Relatório de Processo da Release 2 Sprint 3

Tecnologia: Java | Arquitetura: MVC | Persistência: JSON

Equipe: Eric Natan · José Eudson · Rui Fernando
2026

Tecnologias Utilizadas

- Linguagem de Programação: Java
- Arquitetura: MVC (Model, View, Controller)
- Bibliotecas:
 - java.util
 - java.io
 - JsonUtil (utilitário interno de serialização/desserialização JSON)
- Persistência de dados: JSON
- Ferramenta de build e gerenciamento de dependências: Maven
- Framework de testes: JUnit 5 (junit-jupiter)

Dependências Principais

As dependências do projeto são gerenciadas via Maven e declaradas no arquivo pom.xml. A tabela a seguir lista as bibliotecas utilizadas, suas versões e finalidades:

Dependência	Versão	Uso
Jackson Databind	2.15.2	Serialização/deserialização JSON
Jackson JSR310	2.15.2	Suporte a tipos de data Java 8
JUnit Jupiter	5.10.0	Framework de testes
Mockito	4.11.0	Mocks para testes unitários (compatível com Java 8+)
Byte Buddy	1.17.8	Suporte de instrumentação para JDKs mais novos (ex.: 22/23)
JaCoCo	0.8.10	Cobertura de testes
Maven Shade Plugin	3.2.4	Empacotamento do JAR com dependências

Distribuição de Tarefas e Organização do Trabalho

A equipe manteve a estratégia consolidada nas sprints anteriores de distribuição de tarefas baseada em estimativa de horas por task, visando equilibrar a carga de trabalho entre todos os membros. Antes do início da Sprint 3, a equipe se reuniu para discutir os requisitos funcionais do módulo de matrícula, decompô-los em tasks menores e atribuir a cada task uma estimativa de horas.

Com as estimativas definidas, a distribuição garantiu que o somatório de horas de cada membro ficasse equilibrado, evitando sobrecarga individual. A rastreabilidade foi mantida por meio do Azure DevOps, onde cada requisito funcional foi cadastrado como item de trabalho com link direto para o commit correspondente no GitHub.

Processo de Desenvolvimento e Atribuições

Organização da Equipe — Sprint 3 (02/06/2026 - 11/06/2026)

Gerente da Sprint - Rui Fernando

A Sprint 3 cobre o módulo de matrícula do sistema ClassRoomPB: consulta de turmas disponíveis, solicitação e confirmação automática de matrícula, verificação de pré-requisitos, controle de choque de horário, gerenciamento de lista de espera e cancelamento de matrícula. A tabela a seguir resume a distribuição das responsabilidades entre os membros da equipe.

Membro	RF / Tasks	Entregas
Eric Natan	RF15, RF16, RF17 / Tasks 2.1.1–2.1.6, DOC.S3	Consulta de turmas/disciplinas, verificação de vagas, documentação da sprint, testes
José Eudson	RF21, RF22, RF23 / Tasks 2.1.13–2.1.18	Lista de espera, cancelamento de matrícula, manutenção da lista de espera, testes
Rui Fernando	RF18, RF19, RF20 / Tasks 2.1.7–2.1.12	Verificação de pré-requisitos, choque de horário, confirmação automática de matrícula, testes

Atribuições de Eric Natan (QA):

Consulta de Disciplinas/Turmas Disponíveis (RF15):

- `MatriculaConsultaService`: Serviço responsável por listar as turmas disponíveis para o aluno autenticado, filtrando por período letivo ativo e expondo informações de código, nome da disciplina, professor responsável, horários e vagas restantes.
- `AlunoController`: Extensão do controlador do aluno com a opção de consulta de turmas disponíveis, delegando ao serviço de consulta e exibindo os resultados de forma tabular no console.
- Testes unitários cobrindo listagem de turmas ativas, filtragem correta por período letivo e comportamento quando não há turmas disponíveis.

Solicitação de Matrícula (RF16):

- `MatriculaService`: Serviço central que orquestra o fluxo completo de solicitação de matrícula, invocando as verificações de vagas, pré-requisitos e choque de horário antes de confirmar ou encaminhar o aluno à lista de espera.
- Testes unitários para o fluxo principal de solicitação, cobrindo caminho feliz e cenários de rejeição com mensagens descritivas.

Verificação de Vagas Disponíveis (RF17 — RN03):

- `VagasValidator`: Componente de validação que consulta o número de vagas da turma e compara com o total de matrículas confirmadas, retornando booleano e a quantidade de vagas restantes.
- Integração com `MatriculaService`, que utiliza o `VagasValidator` como primeira etapa do fluxo de matrícula.
- Testes unitários cobrindo turma com vagas disponíveis, turma sem vagas e turma com exatamente uma vaga restante.

Documentação da Sprint (DOC.S3):

- Elaboração do presente relatório de processo da Sprint 3, consolidando as atribuições, decisões técnicas, resultados de testes e análise do burndown.

Atribuições de Rui Fernando (Gerente):

Verificação de Pré-Requisitos (RF18 — RN04 e RN05):

- PreRequisitoValidator: Componente que consulta a lista de pré-requisitos da disciplina alvo e verifica se o aluno possui aprovação registrada em cada uma das disciplinas exigidas.
- Regra RN04: impede matrícula caso algum pré-requisito não tenha sido cursado com aprovação. Regra RN05: disciplinas sem pré-requisitos são sempre liberadas.
- Testes unitários cobrindo aluno com todos os pré-requisitos cumpridos, aluno com pré-requisito pendente, disciplina sem pré-requisitos e aluno sem histórico.

Verificação de Choque de Horário (RF19 — RN02):

- ChoqueHorarioValidator: Componente que recupera os horários das turmas nas quais o aluno já possui matrícula confirmada e verifica sobreposição com os horários da turma solicitada.
- A verificação compara dia da semana, hora de início e hora de fim de cada slot de horário, recusando a matrícula caso haja qualquer interseção.
- Testes unitários para cenários sem sobreposição, sobreposição total, sobreposição parcial e aluno sem matrículas prévias.

Confirmação Automática de Matrícula (RF20):

- Implementação da lógica de confirmação automática no MatriculaService: ao passar por todas as validações (vagas, pré-requisitos e horário), o sistema cria o registro de Matricula com status CONFIRMADA e persiste via MatriculaRepository.
- MatriculaRepository: Repositório responsável pela persistência das matrículas em arquivo JSON (matriculas.json), com suporte a busca por aluno, por turma e por status.
- Testes unitários para confirmação com sucesso, persistência correta do status e recuperação do registro após persistência.

Atribuições de José Eudson (Dev):

Lista de Espera — Entrada (RF21):

- ListaEsperaService: Serviço que gerencia a adição de alunos à fila de espera quando a turma solicitada está sem vagas, atribuindo posição sequencial e registrando data/hora da solicitação.
- ListaEsperaRepository: Repositório responsável pela persistência das filas de espera em arquivo JSON (lista_espera.json).
- Integração com MatriculaService: ao detectar ausência de vagas, o fluxo é redirecionado ao ListaEsperaService em vez de recusar a solicitação.
- Testes unitários para adição com sucesso, posição correta na fila, rejeição de entrada duplicada na mesma turma e fila inicialmente vazia.

Cancelamento de Matrícula (RF22):

- Método cancelarMatricula() no MatriculaService, que verifica se o cancelamento está dentro do período letivo permitido (flag de período letivo ativo) antes de alterar o status da matrícula para CANCELADA.
- Ao confirmar o cancelamento, o serviço verifica automaticamente se há alunos na lista de espera da turma liberada e, caso haja, promove o primeiro da fila para status CONFIRMADA.
- Testes unitários para cancelamento dentro do prazo, tentativa fora do prazo, promoção automática da lista de espera após cancelamento e cancelamento de matrícula inexistente.

Manutenção da Lista de Espera por Turma (RF23):

- Implementação completa do ciclo de vida da lista de espera: consulta por turma, remoção de aluno da fila (desistência), e reordenação automática das posições após remoção ou promoção.
- CoordenadorController: extensão do menu do coordenador com opção de visualizar a lista de espera por turma, exibindo aluno, posição e data de entrada na fila.
- Testes unitários cobrindo consulta de fila por turma, remoção de aluno do meio da fila com reordenação, promoção do primeiro da fila e fila vazia.

Sprint 3 — Burndown

O gráfico de burndown da Sprint 3 reflete o progresso contínuo da equipe ao longo dos dias de trabalho. O escopo total foi dimensionado com base nas 18 tasks (2.1.1 a 2.1.18) mais a task de documentação (DOC.S3), totalizando 19 itens. A linha real de trabalho restante manteve-se próxima da linha ideal, indicando boa aderência ao planejamento da sprint.

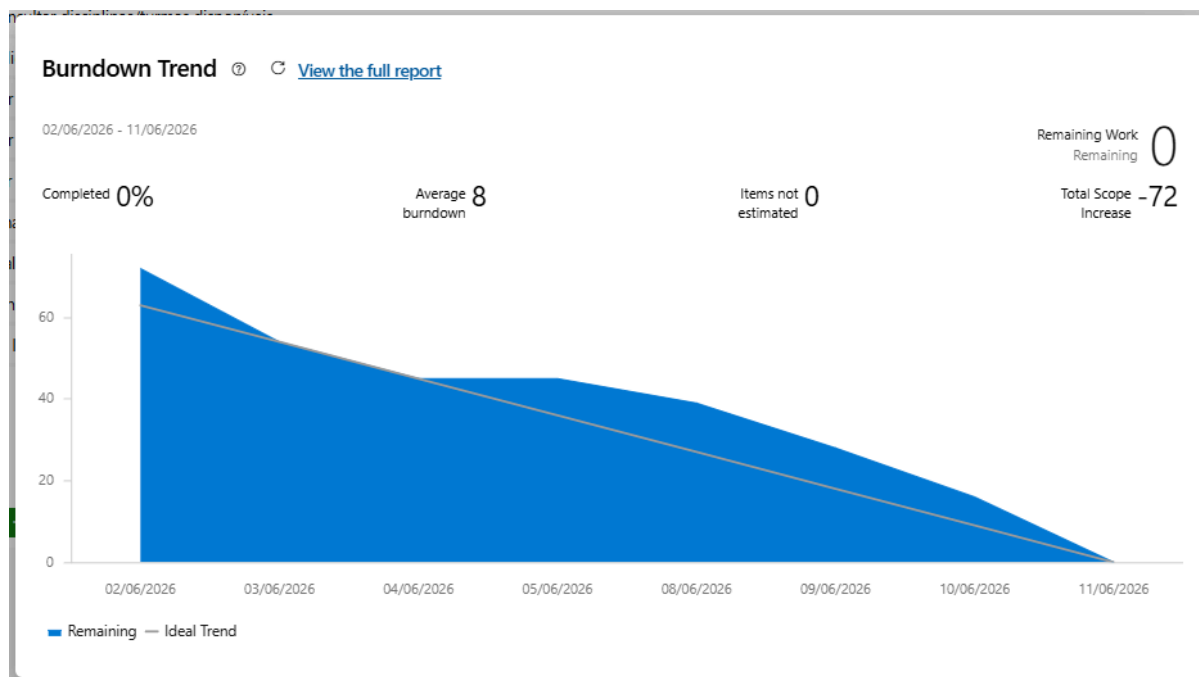


Figura 1: Gráfico de Burndown da Sprint 3

Sprint 3 — Registro das Funcionalidades

A tabela a seguir apresenta o mapeamento entre os requisitos funcionais e as classes implementadas nesta sprint.

RF	Descrição	Responsável
RF15	Consulta de disciplinas/turmas disponíveis pelo Aluno	Eric Natan
RF16	Solicitação de matrícula em uma turma pelo Aluno	Eric Natan
RF17	Verificação de vagas disponíveis (RN03)	Eric Natan
RF18	Verificação de cumprimento de pré-requisitos (RN04, RN05)	Rui Fernando
RF19	Impedimento de choque de horário entre turmas do mesmo Aluno (RN02)	Rui Fernando
RF20	Confirmação automática de matrícula quando todos os critérios são atendidos	Rui Fernando
RF21	Adição do Aluno à lista de espera quando não há vagas	José Eudson
RF22	Cancelamento de matrícula dentro do período permitido	José Eudson
RF23	Manutenção de lista de espera por turma	José Eudson

1. Consulta de Disciplinas/Turmas Disponíveis (RF15)

O sistema disponibiliza ao Aluno autenticado uma listagem de todas as turmas abertas no período letivo ativo. A consulta é realizada pelo `MatriculaConsultaService`, que filtra as turmas com status ativo e retorna, para cada uma, o código e nome da disciplina, o professor responsável, os horários de aula, o número de vagas totais e o número de vagas restantes.

O `AlunoController` foi estendido com a opção de consulta, exibindo os resultados em formato tabular no console. Caso não haja turmas disponíveis no período letivo ativo, uma mensagem informativa é exibida ao aluno sem lançar exceção.

```
=====
                TURMAS DO PERÍODO ATIVO
=====
Período: 2026.1 | 2026-01-01 até 2026-06-02

+-----+-----+-----+-----+-----+-----+
| Código | Disciplina | Horário                | Sala          | Vagas | Professor |
+-----+-----+-----+-----+-----+-----+
| T01    | GP         | Seg 9h-11h / Qua 11h-13h | Bloco A - 118 | 30    | P0001     |
| T01    | EDA        | Seg / Quarta 11h - 13h   | Bloco C - 106 | 45    | P0002     |
+-----+-----+-----+-----+-----+-----+
Total: 2 turma(s).

Pressione ENTER para continuar...
█
```

Figura 2: Listagem de turmas disponíveis para o Aluno

2. Solicitação e Confirmação Automática de Matrícula (RF16 e RF20)

O fluxo de matrícula é centralizado no `MatriculaService`, que executa uma cadeia de validações em sequência: verificação de vagas disponíveis (RF17), cumprimento de pré-requisitos (RF18) e ausência de choque de horário (RF19). Ao passar por todas as etapas com sucesso, a matrícula é automaticamente confirmada (RF20), o registro é criado com status CONFIRMADA e persistido pelo `MatriculaRepository` em arquivo JSON.

Em caso de falha em qualquer validação, o sistema exibe mensagem descritiva informando o motivo da recusa (sem vagas, pré-requisito pendente ou choque de horário) e redireciona o fluxo adequadamente — para a lista de espera em caso de falta de vagas, ou para nova tentativa nos demais casos.

```
=====
                SOLICITAR MATRÍCULA EM TURMA
=====
Período ativo: 2026.1 | 2026-01-01 até 2026-06-02

Código da disciplina: AL█
```

```

=====
                TURMAS DISPONÍVEIS ? AL
=====
+-----+-----+-----+-----+-----+-----+
| Código | Disciplina | Horário   | Sala  | Vagas | Professor |
+-----+-----+-----+-----+-----+-----+
| T01    | AL         | Ter/Qui 7-9h | Lab 3 | 25    | P0002     |
+-----+-----+-----+-----+-----+-----+

Código da turma desejada: T01

[?] Matrícula confirmada com sucesso! Turma: T01 | Disciplina: AL | Período: 2026.1.
Pressione ENTER para continuar...

```

Figura 3: Fluxo de solicitação e confirmação de matrícula

3. Verificação de Vagas Disponíveis (RF17 — RN03)

O VagasValidator consulta o número total de vagas configurado na turma e compara com o total de matrículas com status CONFIRMADA registradas naquela turma. A regra de negócio RN03 determina que a matrícula só pode ser confirmada se houver pelo menos uma vaga disponível. O resultado da validação inclui o número exato de vagas restantes, utilizado tanto para exibição ao aluno quanto para decisão de fluxo no MatriculaService.

4. Verificação de Pré-Requisitos (RF18 — RN04 e RN05)

O PreRequisitoValidator percorre a lista de pré-requisitos da disciplina alvo e, para cada um, verifica no histórico acadêmico do aluno se há registro de aprovação. A regra RN04 impede a matrícula caso qualquer pré-requisito não tenha sido cursado com aprovação. A regra RN05 garante que disciplinas sem pré-requisitos cadastrados sejam sempre liberadas, independentemente do histórico do aluno.

Quando um ou mais pré-requisitos não foram cumpridos, o sistema lista explicitamente quais disciplinas estão pendentes, orientando o aluno sobre o que precisa ser regularizado.

5. Verificação de Choque de Horário (RF19 — RN02)

O ChoqueHorarioValidator recupera todos os horários das turmas nas quais o aluno possui matrícula com status CONFIRMADA e verifica se algum slot da turma solicitada se sobrepõe a qualquer um desses horários. A verificação considera dia da semana, hora de início e hora de fim. Qualquer interseção — total ou parcial — bloqueia a matrícula, e o sistema informa ao aluno quais turmas geram o conflito.


```

=====
                TURMAS DISPONÍVEIS ? EDA
=====
+-----+-----+-----+-----+-----+-----+
| Código | Disciplina | Horário          | Sala          | Vagas | Professor |
+-----+-----+-----+-----+-----+-----+
| T01    | EDA        | Seg / Quarta 7h-9h | Bloco C - 106 | 45    | P0002     |
| T02    | EDA        | Seg / Sex 7h-9h   | Bloco A - 117 | 30    | P0001     |
+-----+-----+-----+-----+-----+-----+

Código da turma desejada: T02

[!] Erro: Existe choque de horário com a turma T01
Pressione ENTER para continuar...

```

Figura 4: Mensagem de choque de horário detectado

6. Lista de Espera e Cancelamento de Matrícula (RF21, RF22 e RF23)

Quando uma turma está sem vagas, o fluxo de matrícula redireciona automaticamente o aluno para a lista de espera (RF21), gerenciada pelo ListaEsperaService. O aluno recebe uma posição sequencial na fila e uma confirmação com a posição atual. A lista é persistida pelo ListaEsperaRepository em arquivo JSON, garantindo que a ordem seja preservada mesmo após reinicializações do sistema.

O cancelamento de matrícula (RF22) só é permitido dentro do período letivo ativo. Ao cancelar, o sistema verifica automaticamente se há alunos aguardando na lista de espera da turma liberada. Em caso afirmativo, o primeiro da fila é promovido para status CONFIRMADA sem necessidade de nova ação do aluno.

O RF23 complementa o ciclo com a manutenção completa da lista de espera: consulta por turma (disponível ao Coordenador), remoção voluntária do aluno da fila e reordenação automática das posições sempre que um aluno é promovido ou se retira.

Testes Unitários

Todos os requisitos implementados nesta sprint possuem cobertura de testes unitários, escritos com o framework JUnit 5 (junit-jupiter) e executados via Maven (mvn test). Os testes foram organizados em classes aninhadas (@Nested) para separar claramente os cenários de cada funcionalidade.

Testes de Consulta e Matrícula (RF15, RF16 e RF17)

Os testes utilizam @TempDir do JUnit 5 para isolar arquivos de persistência por execução. Os cenários cobertos incluem:

- Consulta de turmas: listagem correta para período ativo, ausência de turmas de outros períodos, comportamento com período sem turmas.
- Solicitação de matrícula: caminho feliz com todos os critérios atendidos, rejeição com mensagem adequada em cada cenário de falha.
- Verificação de vagas: turma com vagas, sem vagas e com exatamente uma vaga restante; consistência após confirmação de matrícula.

Testes de Pré-Requisitos e Choque de Horário (RF18 e RF19)

Os cenários cobertos incluem:

- Pré-requisitos: aluno com histórico completo, aluno com pré-requisito pendente (listagem dos pendentes), disciplina sem pré-requisitos, aluno sem histórico acadêmico.
- Choque de horário: sem sobreposição, sobreposição total de slot, sobreposição parcial de horário, aluno sem matrículas anteriores.

Testes de Lista de Espera e Cancelamento (RF20, RF21, RF22 e RF23)

Os cenários cobertos incluem:

- Confirmação automática: persistência correta do status CONFIRMADA, decremento de vaga disponível, recuperação do registro pós-persistência.
- Lista de espera: adição com posição correta, rejeição de entrada duplicada, fila vazia, múltiplos alunos na fila com ordenação preservada.
- Cancelamento: dentro do prazo, fora do prazo (rejeição), promoção automática do primeiro da fila, turma sem lista de espera após cancelamento.
- Manutenção da lista: consulta por turma, remoção do meio com reordenação, promoção do primeiro e verificação de consistência da fila resultante.

Cobertura de Testes (JaCoCo)

O relatório de cobertura gerado pelo JaCoCo ao final da Sprint 3 manteve o padrão de qualidade das sprints anteriores, com cobertura global acima de 90% de instruções e branches. A figura abaixo exibe o resumo por pacote:

classroompb

classroompb

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.classroompb.service		87%		77%	118	387	90	711	7	92	0	10
com.classroompb.repository		92%		83%	13	122	23	276	4	94	1	7
com.classroompb.model		91%		83%	9	110	14	185	8	104	0	13
com.classroompb.util		91%		85%	5	18	3	34	2	8	0	2
com.classroompb.exception		100%		100%	0	6	0	11	0	5	0	2
Total	533 of 5,021	89%	144 of 668	78%	145	643	130	1,217	21	303	1	34

Figura 5: Relatório de cobertura de testes — JaCoCo (Sprint 3)

Persistência de Dados

O sistema ClassRoomPB mantém o JSON como mecanismo de persistência de dados. Nesta sprint, dois novos arquivos de persistência foram introduzidos para suportar o módulo de matrícula:

- matriculas_turma.json: armazena os registros de matrícula com matrícula do aluno, código da turma, status (CONFIRMADA ou CANCELADA) e data de solicitação.
- lista_espera.json: armazena as filas de espera por turma, com matrícula do aluno, posição na fila e data/hora de entrada.

Os arquivos existentes das sprints anteriores (usuarios.json, cursos.json, disciplinas.json e periodos_letivos.json) foram preservados sem alterações estruturais, mantendo a compatibilidade com as funcionalidades já entregues.

```
{ } matriculas_turma.json > ...
1  [ {
2    "matriculaAluno" : "A0001",
3    "codigoDisciplina" : "EDA",
4    "codigoPeriodo" : "2026.1",
5    "codigoTurma" : "T01",
6    "status" : "CANCELADA",
7    "dataSolicitacao" : "2026-06-03T18:36:12.0307268"
8  }, {
9    "matriculaAluno" : "A0001",
10   "codigoDisciplina" : "GP",
11   "codigoPeriodo" : "2026.1",
12   "codigoTurma" : "T01",
13   "status" : "CONFIRMADA",
14   "dataSolicitacao" : "2026-06-03T18:36:46.3637582"
15 }, {
16   "matriculaAluno" : "A0001",
17   "codigoDisciplina" : "EDA",
18   "codigoPeriodo" : "2026.1",
19   "codigoTurma" : "T01",
20   "status" : "PENDENTE",
21   "dataSolicitacao" : "2026-06-04T10:41:43.3438611"
22 }, {
23   "matriculaAluno" : "A0001",
24   "codigoDisciplina" : "AL",
25   "codigoPeriodo" : "2026.1",
26   "codigoTurma" : "T01",
27   "status" : "CONFIRMADA",
28   "dataSolicitacao" : "2026-06-11T17:00:27.1885395"
29 } ]
```

Figura 6: Conteúdo do arquivo `matriculas_turma.json`